

AWS SAA-C03



PROJECTS

★ BUILD • DEPLOY • SECURE • SCALE • MONITOR • OPTIMIZE ★



CLOUD
ARCHITECTURE
PROJECTS



SECURE
BY
DESIGN



SCALABLE
& RELIABLE
SOLUTIONS



MONITOR
& OPERATE
WITH CONFIDENCE



REAL WORLD
SCENARIOS &
BEST PRACTICES



WHAT YOU WILL BUILD

- ✓ HIGHLY AVAILABLE APPLICATIONS
- ✓ SERVERLESS SOLUTIONS
- ✓ SECURE INFRASTRUCTURE
- ✓ DATA STORAGE & MANAGEMENT
- ✓ EVENT-DRIVEN SYSTEMS
- ✓ NETWORKING & CONNECTIVITY
- ✓ MONITORING & OBSERVABILITY
- ✓ DISASTER RECOVERY

20 PRACTICAL PROJECTS.
REAL AWS. REAL EXPERIENCE. REAL IMPACT.



PROJECT 1

BUILD A HIGHLY AVAILABLE WEB APPLICATION

DESIGN ACROSS TWO AVAILABILITY ZONES FOR HIGH AVAILABILITY AND RESILIENCE

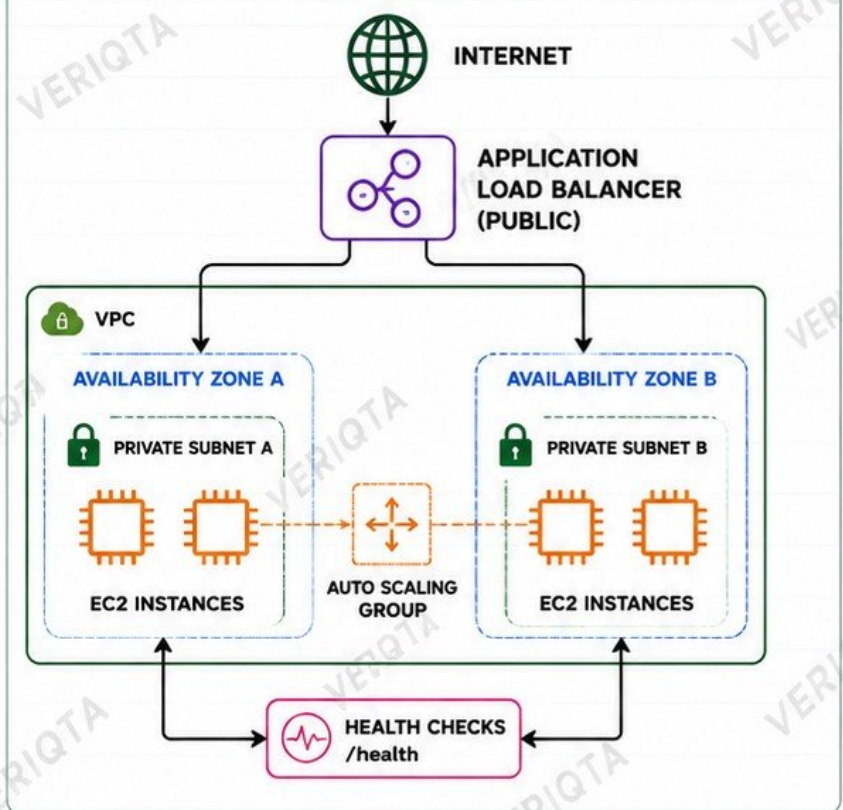
WHAT YOU WILL BUILD

- ✓ Design across two Availability Zones
- ✓ Public Application Load Balancer
- ✓ EC2 instances in private subnets
- ✓ Auto Scaling Group for the app tier
- ✓ Health checks for reliability
- ✓ Security groups for traffic control
- ✓ Test instance failure and recovery
- ✓ Highly available and resilient web app

KEY SERVICES USED

-  APPLICATION LOAD BALANCER
-  EC2 INSTANCES
-  AUTO SCALING GROUP
-  SECURITY GROUPS
-  HEALTH CHECKS
-  AVAILABILITY ZONES

ARCHITECTURE OVERVIEW



HOW IT WORKS

- 1 Users request your application over the Internet.
- 2 Traffic hits the Application Load Balancer (ALB).
- 3 ALB forwards requests to healthy EC2 instances in both Availability Zones.
- 4 Auto Scaling Group maintains the desired capacity across AZs.
- 5 Health checks ensure only healthy instances receive traffic.
- 6 If an instance fails, traffic is automatically routed to healthy instances and a new instance is launched.

TEST FAILURE AND RECOVERY

- 1 SSH into one EC2 instance and stop the application (or terminate the instance).
- 2 Observe the instance become unhealthy.
- 3 ALB stops sending traffic to the unhealthy instance.
- 4 Auto Scaling Group launches a new instance in the same or other AZ.
- 5 ALB routes traffic to the new healthy instance.
- 6 Your application stays available!



JUNIOR ENGINEER TIP

High availability is not just multiple servers. It is proper architecture, health checks, and automatic recovery.

PROJECT 2

DESIGN A PRODUCTION VPC

BUILD A SECURE, SCALABLE, AND WELL-ARCHITECTED VPC FOR PRODUCTION WORKLOADS

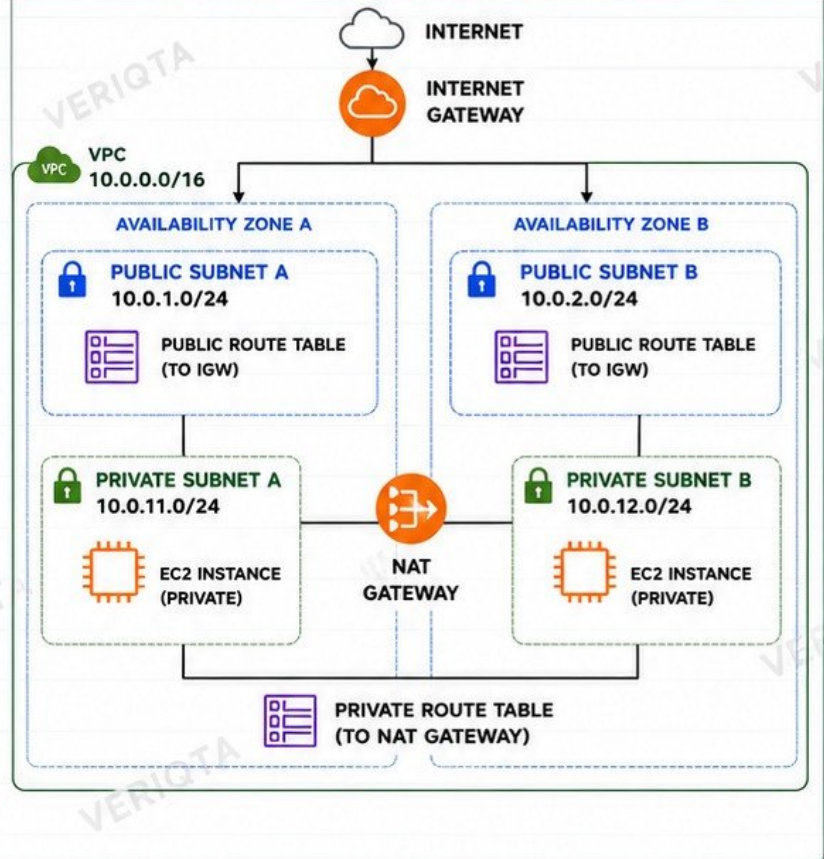
WHAT YOU WILL BUILD

- ✓ Public and private subnets across multiple AZs
- ✓ Internet Gateway for public access
- ✓ NAT Gateway for private subnet Internet access
- ✓ Route tables for proper traffic flow
- ✓ Security Groups vs Network ACLs
- ✓ Verify public and private routing
- ✓ Troubleshoot private instance with no outbound Internet

KEY COMPONENTS

- VPC
- PUBLIC SUBNET
- PRIVATE SUBNET
- INTERNET GATEWAY
- NAT GATEWAY
- ROUTE TABLES
- SECURITY GROUPS
- NETWORK ACLS

HIGH-LEVEL VPC ARCHITECTURE



ROUTE TABLES

PUBLIC ROUTE TABLE

- 10.0.0.0/16 → local
- 0.0.0.0/0 → IGW

PRIVATE ROUTE TABLE

- 10.0.0.0/16 → local
- 0.0.0.0/0 → NAT Gateway

SECURITY GROUPS vs NETWORK ACLS

SECURITY GROUPS (STATEFUL)

- ✓ Operate at instance level
- ✓ Allow rules only
- ✓ Return traffic allowed automatically
- ✓ Example: Allow HTTP (80) from 0.0.0.0/0

NETWORK ACLS (STATELESS)

- ✗ Operate at subnet level
- ✗ Allow and Deny rules
- ✗ Return traffic must be explicitly allowed
- ✗ Example: Allow 80 in and 80 out

VERIFY ROUTING

- ✓ Public instance can access Internet
- ✓ Private instance can access Internet (via NAT)
- ✓ Private instance cannot be reached directly from Internet

TROUBLESHOOT: PRIVATE INSTANCE HAS NO OUTBOUND INTERNET ACCESS

STEP	CHECK	COMMAND / ACTION	WHAT TO LOOK FOR
1	Route Table	Check route table associated with private subnet	Must have 0.0.0.0/0 route to NAT Gateway
2	NAT Gateway	Check NAT Gateway status	Must be in AVAILABLE state
3	Subnet Association	Ensure private subnet is associated with correct route table	Correct route table attached
4	Security Group	Check outbound rules of instance SG	Allow outbound traffic (usually 0.0.0.0/0)
5	NACL	Check NACL rules for subnet	Allow outbound (ephemeral ports 1024-65535)
6	DNS	Test DNS resolution	Use nslookup or dig from instance



JUNIOR ENGINEER TIP

Most "no Internet" issues in private subnets are caused by:

- Missing route to NAT
- NACL blocking outbound
- NAT Gateway not in the same AZ
- Wrong route table association



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PROJECT 3

HOST A SECURE STATIC WEBSITE

BUILD A FAST, SECURE, AND HIGHLY AVAILABLE STATIC WEBSITE ON AWS

WHAT YOU WILL BUILD

- ✓ Amazon S3 for static website files
- ✓ Block Public Access (keep bucket private)
- ✓ CloudFront as the CDN
- ✓ Origin Access Control (OAC)
- ✓ HTTPS with ACM certificate
- ✓ Route 53 for custom domain
- ✓ Cache behavior for performance
- ✓ A secure, production-ready static website

ARCHITECTURE OVERVIEW



User → Route 53 → CloudFront → Private S3 Bucket

KEY AWS SERVICES

-  **AMAZON S3**
Stores website files
-  **CLOUDFRONT**
Global CDN
-  **ROUTE 53**
Domain & DNS
-  **ACM**
SSL/TLS certificate
-  **ORIGIN ACCESS CONTROL**
Secure access to S3
-  **CACHE BEHAVIOR**
Performance & caching

STEP-BY-STEP BUILD

- 1 Create an S3 bucket for your static website files.
- 2 Enable Block Public Access (keep the bucket private).
- 3 Upload your website files to S3.
- 4 Create an ACM certificate (in us-east-1 for CloudFront).
- 5 Create a CloudFront distribution.
- 6 Configure Origin Access Control (OAC) and attach it to your S3 bucket.
- 7 Set up cache behavior (default and optional additional behaviors).
- 8 Add your custom domain in Route 53.
- 9 Test HTTPS access to your website.
- 10 Verify that the S3 bucket is not publicly accessible.
- 11 Test performance and caching (check cache headers).
- 12 Confirm your website is secure, fast, and working globally.

IMPORTANT SECURITY POINTS

-  Keep your S3 bucket private.
-  Use Origin Access Control (OAC) instead of the older OAI.
-  Enforce HTTPS with ACM certificate.
-  Do not expose your S3 bucket to the public.
-  Use least privilege bucket policy for CloudFront.

JUNIOR ENGINEER TIP

-  Always use CloudFront + OAC for a secure, scalable, and production-ready static website. Never make your S3 bucket public.

EXAMPLE S3 BUCKET POLICY (WITH OAC)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "cloudfront.amazonaws.com" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::your-bucket-name/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/EXAMPLE"
        }
      }
    }
  ]
}
  
```

NEXT STEPS

- ✓ Add custom error pages (404, 403).
- ✓ Enable access logs in CloudFront.
- ✓ Monitor with CloudWatch.
- ✓ Explore invalidations and cache TTL settings.

PROJECT 4

BUILD A HIGHLY AVAILABLE DATABASE ARCHITECTURE

DESIGN A RESILIENT, SECURE, AND HIGHLY AVAILABLE DATABASE ON AWS

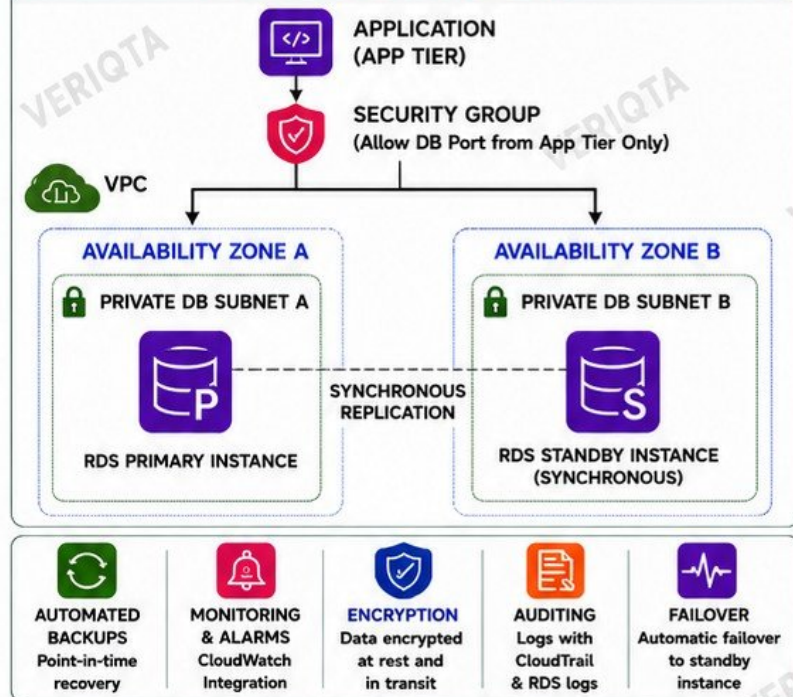
WHAT YOU WILL BUILD

- ✓ Amazon RDS for managed databases
- ✓ Private database subnets
- ✓ Multi-AZ deployment
- ✓ Security groups for controlled access
- ✓ Automated backups
- ✓ Failover for disaster tolerance
- ✓ Application to database connection architecture
- ✓ Understand Multi-AZ vs Read Replicas

KEY AWS SERVICES USED

- AMAZON RDS
- VPC
- PRIVATE SUBNETS
- SECURITY GROUPS
- AWS BACKUP
- AMAZON CLOUDWATCH

ARCHITECTURE OVERVIEW



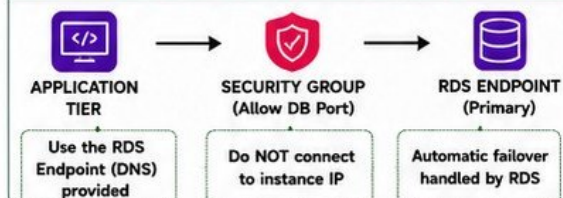
RDS MULTI-AZ (HIGH AVAILABILITY)

- ✓ One primary DB instance in AZ A
 - ✓ One synchronous standby instance in AZ B
 - ✓ Automatic failover to standby if primary fails
 - ✓ Same compute size as primary
 - ✓ Ideal for HA and failover within the same region
- 💡 **BEST FOR:** High Availability & Disaster Tolerance

RDS READ REPLICAS (SCALING READS)

- ✓ One primary DB instance (writes)
 - ✓ One or more read replicas (async replication)
 - ✓ Read replicas in same or different AZ/Region
 - ✓ Improve read performance and offload reads
 - ✓ Not used for automatic failover
- 💡 **BEST FOR:** Scaling Read Workloads

CONNECTION ARCHITECTURE



AUTOMATED BACKUPS

- ✓ Automated daily backups with retention (1-35 days)
- ✓ Point-in-time recovery to any second
- ✓ Stored in durable Amazon S3
- ✓ Test restore regularly

FAILOVER PROCESS (MULTI-AZ)

- 1 Primary instance fails
- 2 RDS detects failure
- 3 Standby instance promoted to primary
- 4 DNS endpoint updated automatically
- 5 Application reconnects automatically

MULTI-AZ vs READ REPLICAS

FEATURE	MULTI-AZ (HA)	READ REPLICAS (SCALING)
Purpose	High Availability & Failover	Scale Read Workloads
Replication	Synchronous	Asynchronous
Failover	Automatic	Manual (promote replica)
Standby	One standby	One or more replicas
Compute Cost	~2x of primary	Additional cost per replica
Best For	Mission-critical apps	Reporting, analytics, heavy reads



JUNIOR ENGINEER TIP

- ✓ Always use private subnets for databases.
- ✓ Restrict access using Security Groups.
- ✓ Enable automated backups and test restores.
- ✓ Use Multi-AZ for HA. Use Read Replicas for scaling.
- ✓ Use the RDS endpoint, not instance IP.
- ✓ Monitor failover and connection recovery.



@veriqta



@veriqta



veriqta



veriqta



@veriqta



@veriqta

PROJECT 5

SCALE A READ-HEAVY APPLICATION

USE READ REPLICAS AND CACHING TO IMPROVE PERFORMANCE AND SCALE

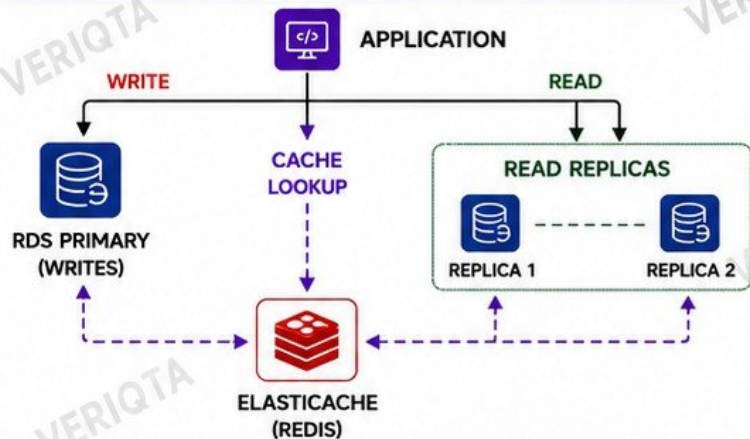
WHAT YOU WILL BUILD

- ✓ RDS Read Replicas for scaling reads
- ✓ Read/Write separation in the app
- ✓ Amazon ElastiCache (Redis) for caching
- ✓ Cache hit and miss flow
- ✓ Identify and reduce database bottlenecks
- ✓ Choose the right scaling strategy: Vertical, Replicate, or Cache
- ✓ Monitor and test performance

KEY AWS SERVICES USED

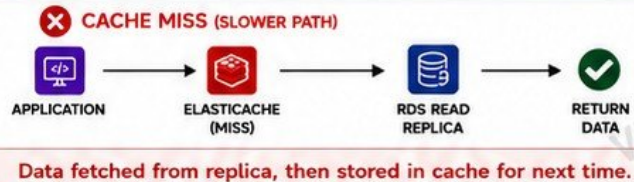
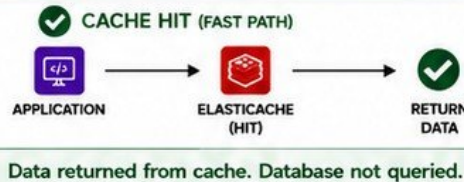
- AMAZON RDS
- RDS READ REPLICAS
- AMAZON ELASTICACHE (REDIS)
- AMAZON CLOUDWATCH
- AWS IAM

ARCHITECTURE OVERVIEW



i Reads are served from cache first. If miss, read from replicas. Writes go to the primary. Replicas stay in sync asynchronously.

CACHE HIT AND MISS FLOW



IDENTIFYING DATABASE BOTTLENECKS

- ✓ High CPU on DB instances
- ✓ High number of read requests
- ✓ Slow queries / missing indexes
- ✓ Connection exhaustion
- ✓ Large unoptimized queries
- ✓ High disk I/O or storage latency

SCALING STRATEGY: WHEN TO USE WHAT?

SCALE VERTICALLY



- Increase DB instance size
- More CPU, RAM, IOPS
- Use for overall load increase

Best for: Write-heavy workloads

SCALE WITH REPLICAS



- Add Read Replicas
- Distribute read traffic
- Improves read scalability

Best for: Read-heavy workloads

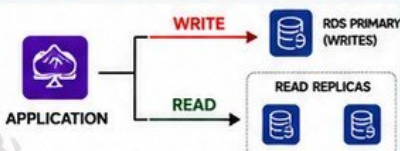
USE CACHING



- Store frequent data in cache
- Reduce DB load
- Microsecond responses

Best for: Repeated / hot data

READ/WRITE SEPARATION PATTERN



COMMON DATABASE BOTTLENECKS AND FIXES

BOTTLENECK	HOW TO IMPROVE
High CPU	Optimize queries, Add Replicas, Scale up
High Read Load	Add Read Replicas, Implement caching
Slow Queries	Add indexes, Optimize SQL
Connection Limits	Use connection pooling, Increase limits
High Disk I/O	Use faster storage, Optimize queries

PERFORMANCE MONITORING (CLOUDWATCH)



DECISION GUIDE

SITUATION	BEST SOLUTION	WHY?
High reads, low writes	Add Read Replicas + Cache	Offload reads, reduce DB load
Slow response for repeated data	Use ElastiCache (Redis)	Serve hot data from memory
High overall load (reads + writes)	Scale Vertically	More resources for everything
Read replicas lagging	Improve queries / Add replicas	Reduce load per replica
Cost too high	Combine cache + replicas	Balance performance and cost



JUNIOR ENGINEER TIP

- ✓ Cache the data that doesn't change often.
- ✓ Always separate reads from writes.
- ✓ Monitor before you scale.
- ✓ Start with replicas, add cache for hotspots.
- ✓ Test failure: stop a replica and confirm traffic still flows.

PROJECT 6

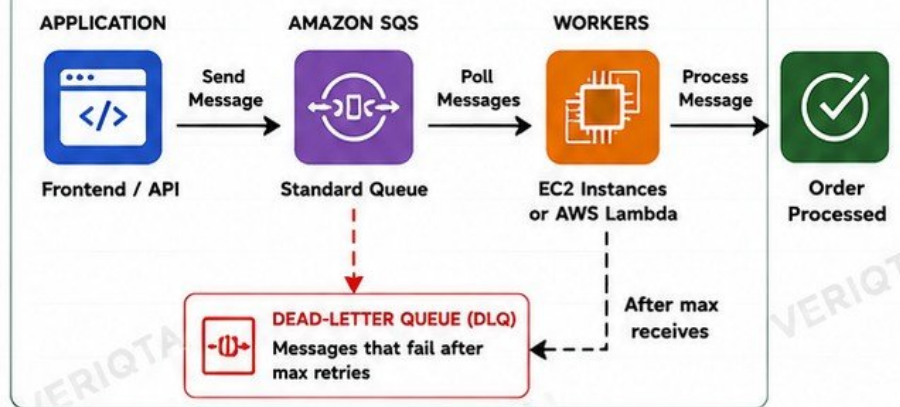
BUILD A DECOUPLED ORDER PROCESSING SYSTEM

USE AMAZON SQS TO DECOUPLE YOUR APPLICATION FROM BACKGROUND PROCESSING

WHAT YOU WILL BUILD

- ✓ Frontend/API accepts orders
- ✓ Messages are sent to Amazon SQS
- ✓ Workers (EC2 or Lambda) process orders
- ✓ Visibility timeout prevents duplicate work
- ✓ Dead-letter queue captures failed messages
- ✓ Retry behavior handled automatically
- ✓ System remains stable during failures

ARCHITECTURE OVERVIEW



HOW IT WORKS

- 1 Application sends order to SQS.
- 2 SQS stores the message durably.
- 3 Workers poll the queue and process messages.
- 4 If processing succeeds, the message is deleted.
- 5 If processing fails, the message becomes visible again after the visibility timeout.
- 6 After the max receive count, the message goes to the DLQ. Your app stays responsive!

VISIBILITY TIMEOUT



When a worker picks up a message, it becomes invisible to other workers for a period of time.

WHY IT MATTERS

- Prevents multiple workers from processing the same message.
- If processing takes longer than the timeout, the message becomes visible again and may be picked up by another worker.

RETRY & DEAD-LETTER QUEUE



RETRY BEHAVIOR

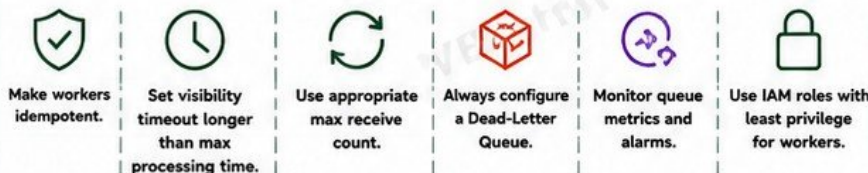
- SQS automatically retries a failed message.
- The message returns to the queue after the visibility timeout expires.
- Default max receive count is set on the redrive policy.



DEAD-LETTER QUEUE (DLQ)

- Captures messages that fail processing repeatedly.
- Prevents poison messages from blocking the main queue.
- Allows inspection and manual reprocessing.

BEST PRACTICES



KEY SQS TERMS

QUEUE: Holds messages.
PRODUCER: Sends messages.
CONSUMER: Receives and processes messages.
VISIBILITY TIMEOUT: Time a message remains hidden after being received.
DLQ: Captures failed messages after max retries.



JUNIOR ENGINEER TIP

Decoupling with SQS improves reliability and scalability. Your application can continue to accept orders even if workers or downstream services are temporarily down.

SIMPLE MESSAGE FLOW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DECOUPLE. PROCESS. RECOVER. DELIVER RELIABLE SYSTEMS.

PROJECT 7

BUILD AN EVENT-DRIVEN APPLICATION

BUILD SCALABLE, ASYNCHRONOUS APPLICATIONS USING AWS EVENTS SERVICES

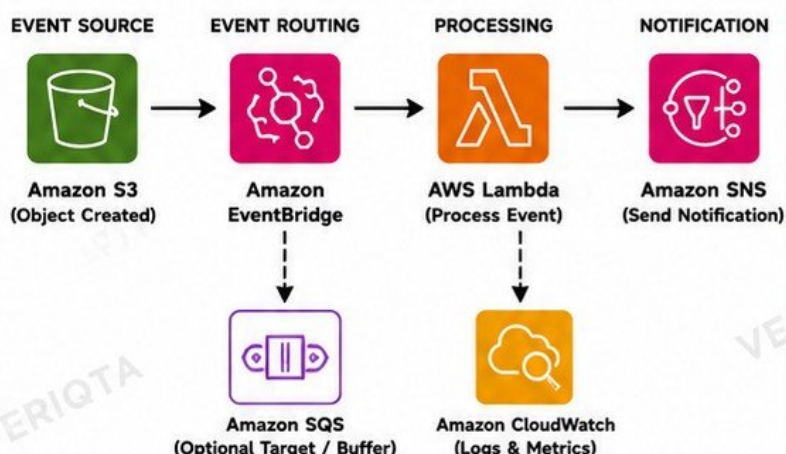
WHAT YOU WILL BUILD

- ✓ Use Amazon S3 events as the trigger
- ✓ Route events using Amazon EventBridge
- ✓ Process events with AWS Lambda
- ✓ Notify users via Amazon SNS
- ✓ Understand event routing and filtering
- ✓ Enable asynchronous processing
- ✓ Handle failures and retries
- ✓ Compare SNS, SQS, and EventBridge

HOW IT WORKS

- 1 A file is uploaded to an S3 bucket.
- 2 S3 emits an event.
- 3 EventBridge receives the event and matches rules.
- 4 EventBridge routes the event to the target(s).
- 5 Lambda processes the event asynchronously.
- 6 SNS sends notifications to subscribers.

ARCHITECTURE OVERVIEW



EVENT FLOW (DETAIL)



FAILURE HANDLING

- Retries**
Lambda automatically retries on failure.
- Dead-Letter Queue (DLQ)**
Failed messages can be sent to an SQS DLQ.
- Monitoring**
CloudWatch Alarms notify you of failures.
- Idempotency**
Design your Lambda to be idempotent.

COMPARE: SNS vs SQS vs EVENTBRIDGE

FEATURE	AMAZON SNS	AMAZON SQS	AMAZON EVENTBRIDGE
Purpose	Pub/Sub messaging service	Message queue service	Event routing and bus service
Delivery	Push (to subscribers)	Pull (by consumers)	Push (to targets)
Communication	One-to-Many	One-to-One	One-to-Many
Persistence	No (not stored)	Yes (stored until consumed)	No (events routed in real-time)
Ordering	No ordering	FIFO queue supports ordering	No ordering
Best Use Case	Notifications, Alerts	Decoupling, Buffering, Work queues	Event routing, Serverless workflows



JUNIOR ENGINEER TIP

Event-driven architectures improve scalability and decouple components. Always design for failures, retries, and observability.

COMMON USE CASES

- ✓ File processing pipelines
- ✓ User sign-up notifications
- ✓ Image resizing & thumbnails
- ✓ Audit logging & alerts
- ✓ Data sync between services
- ✓ Real-time analytics triggers



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

EVENTS DRIVE AUTOMATION. AUTOMATION DRIVES INNOVATION.

PROJECT 8

SECURE APPLICATION ACCESS WITH IAM

USE IAM TO SECURE AWS RESOURCES AND ELIMINATE LONG-TERM CREDENTIALS

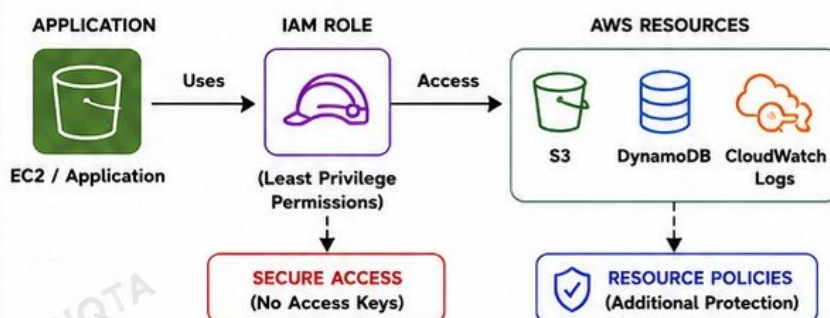
WHAT YOU WILL BUILD

- ✓ Create IAM roles for secure access
- ✓ Attach roles to EC2 using instance profiles
- ✓ Use resource policies to control access
- ✓ Apply least privilege principles
- ✓ Use temporary credentials (STS)
- ✓ Remove hardcoded access keys
- ✓ Test denied and permitted actions
- ✓ Troubleshoot AccessDenied errors

KEY IAM CONCEPTS

-  **IAM ROLES**
Trusted identity with permissions
-  **INSTANCE PROFILES**
Attach roles to EC2 instances
-  **RESOURCE POLICIES**
Control access at the resource level
-  **LEAST PRIVILEGE**
Grant only the minimum required permissions
-  **TEMPORARY CREDENTIALS**
Short-lived credentials improve security

ARCHITECTURE OVERVIEW



HOW IT WORKS

- 1 EC2 instance starts and assumes an IAM role via the instance profile.
- 2 The application gets temporary credentials from AWS STS.
- 3 Application makes requests to AWS services.
- 4 IAM policies attached to the role allow or deny the actions.
- 5 Resource policies provide an additional layer of security.
- 6 No long-term credentials are stored on the instance or in the code.

TEST ACCESS

✓ PERMITTED ACTION (Example)

```
aws s3 ls s3://my-app-bucket
```

Expected: Success

✗ DENIED ACTION (Example)

```
aws s3 rm s3://my-app-bucket/file.txt
```

Expected: AccessDenied

TROUBLESHOOT ACCESSDENIED

- 1 Check the error message for the denied action.
- 2 Verify the IAM role attached to the instance.
- 3 Review the IAM policy attached to the role.
- 4 Check resource policies for explicit denies.
- 5 Use AWS CloudTrail to see the exact API call.
- 6 Test with `aws sts get-caller-identity` to confirm identity.

BEST PRACTICES

-  **FOLLOW LEAST PRIVILEGE**
-  **USE TEMPORARY CREDENTIALS (STS)**
-  **NEVER STORE ACCESS KEYS IN CODE**
-  **USE RESOURCE POLICIES FOR EXTRA SECURITY**
-  **MONITOR & AUDIT ACCESS WITH CLOUDTRAIL**

EXAMPLE IAM POLICY (S3 READ ONLY)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::my-app-bucket"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::my-app-bucket/*"
    }
  ]
}
```

REMOVE HARDCODED KEYS



DO NOT hardcode access keys in your application or scripts. Use IAM roles and temporary credentials instead.



JUNIOR ENGINEER TIP

If you get AccessDenied, it's not AWS being difficult. It's AWS protecting your resources. Fix the policy, not the security.

COMMON USE CASES

- ✓ EC2 applications accessing S3, DynamoDB, or RDS
- ✓ Lambda functions accessing AWS services
- ✓ Cross-account access with STS
- ✓ CI/CD pipelines
- ✓ Access for container tasks
- ✓ Monitoring and logging

PROJECT 9

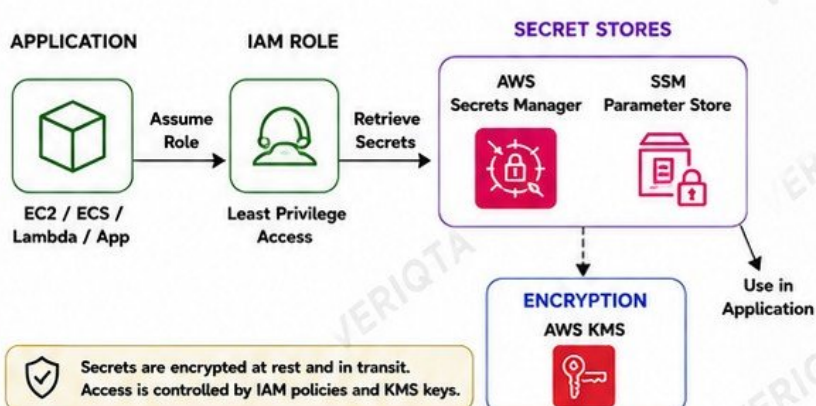
STORE AND RETRIEVE APPLICATION SECRETS SAFELY

USE AWS SECRETS MANAGER, SSM PARAMETER STORE AND KMS TO PROTECT SECRETS

WHAT YOU WILL BUILD

- ✓ Store secrets using AWS Secrets Manager and SSM Parameter Store
- ✓ Encrypt secrets using AWS KMS
- ✓ Retrieve secrets securely using an application IAM role
- ✓ Understand secret rotation concepts
- ✓ Compare Secrets Manager vs Parameter Store
- ✓ Follow least privilege access
- ✓ Remove hardcoded secrets from applications

ARCHITECTURE OVERVIEW



HOW IT WORKS

1. Store secrets in AWS Secrets Manager or SSM Parameter Store.
2. Secrets are encrypted using AWS KMS.
3. Application assumes an IAM role.
4. Application requests the secret securely.
5. AWS service decrypts and returns the secret.
6. Application uses the secret in memory (not stored on disk).
7. Rotate secrets regularly and monitor access.

SECRET RETRIEVAL EXAMPLE

Retrieve from Secrets Manager (CLI)

```
aws secretsmanager get-secret-value \
  --secret-id my/db/secret
```

Retrieve from Parameter Store (CLI)

```
aws ssm get-parameter \
  --name /myapp/db/password \
  --with-decryption
```

Never hardcode secrets in code, config files, or environment variables.

ROTATION CONCEPTS



Rotation automatically updates secrets to reduce risk.

- ✓ Secrets Manager supports built-in rotation for many services.
- ✓ Custom rotation with Lambda for other databases and APIs.
- ✓ Rotation steps: create -> set -> test -> finish.
- ✓ Applications should use the latest secret version.

SECRETS MANAGER vs SSM PARAMETER STORE

FEATURE	AWS SECRETS MANAGER	SSM PARAMETER STORE
Purpose	Store, manage and rotate secrets	Store configuration and secrets
Best For	Database credentials, API keys, rotation requirements	App configs, environment variables, non-rotating secrets
Rotation	Built-in rotation support	No built-in rotation
Cost	Higher (per secret + API calls)	Lower (especially Standard tier)
Security	Automatic encryption, fine-grained access, audit	Secure, but more manual management
Size Limit	Up to 64 KB per secret	Up to 8 KB (Standard), higher in Advanced
Versions	Stores multiple secret versions	Supports versions for parameters
Use When	You need rotation, auditing, rich features	You need low-cost config or simple secrets

KEY SERVICES



AWS SECRETS MANAGER
Manage, rotate and retrieve secrets securely.



SSM PARAMETER STORE
Store hierarchical parameters and secure strings.



AWS KMS
Encrypts secrets and controls encryption keys.



JUNIOR ENGINEER TIP

Always use IAM roles and KMS encryption. Never hardcode secrets. Use the service designed for your use case.

COMMON MISTAKES

- ✗ Hardcoding secrets in code or configs
- ✗ Over-permissive IAM policies
- ✗ Not enabling rotation
- ✗ Storing large configs in Secrets Manager
- ✗ Ignoring access logs and monitoring
- ✗ Using the wrong service for the use case



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SECURE SECRETS. PROTECT APPLICATIONS. EARN TRUST.

PROJECT 10

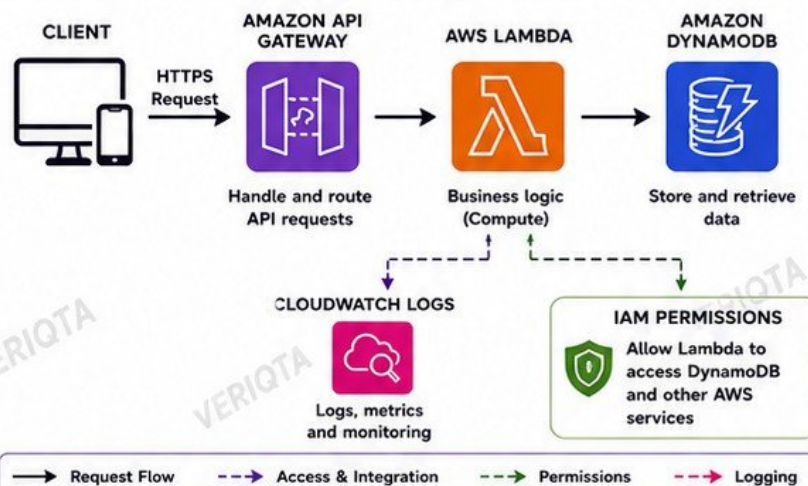
BUILD A SERVERLESS REST API

BUILD A SECURE, SCALABLE AND COST-EFFECTIVE SERVERLESS API ON AWS

WHAT YOU WILL BUILD

- ✓ Create a REST API using Amazon API Gateway
- ✓ Process requests with AWS Lambda
- ✓ Store and retrieve data in DynamoDB
- ✓ Secure access with IAM permissions
- ✓ Log and monitor with CloudWatch Logs
- ✓ Understand the request flow end-to-end
- ✓ Implement error handling and responses
- ✓ Deploy a fully serverless architecture

ARCHITECTURE OVERVIEW



HOW IT WORKS

- 1 Client sends an HTTP request to the API Gateway.
- 2 API Gateway validates the request and routes it to the appropriate Lambda function.
- 3 Lambda processes the request and performs operations on DynamoDB.
- 4 Lambda returns a response back to API Gateway.
- 5 API Gateway sends the response back to the client.
- 6 All logs and errors are sent to CloudWatch Logs.
- 7 IAM roles ensure secure access between services.

REQUEST EXAMPLE

GET /items

Headers:
Authorization: Bearer <token>

Response: 200 OK

```
{
  "items": [
    {
      "id": "1",
      "name": "Sample Item"
    }
  ]
}
```

ERROR HANDLING

- ❗ Validate all inputs.
- ❗ Return meaningful HTTP status codes.
- ❗ Use try/catch in Lambda functions.
- ❗ Log errors to CloudWatch.
- ❗ Use custom error messages for clients.

COMMON STATUS CODES

200	Success
400	Bad Request
401	Unauthorized
500	Internal Server Error

IAM PERMISSIONS (EXAMPLE)

Lambda Execution Role Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:DeleteItem"
      ],
      "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/MyTable"
    }
  ]
}
```

API ENDPOINT EXAMPLES

METHOD	ENDPOINT	DESCRIPTION	INTEGRATION	DYNAMODB ACTION
GET	/items	Get all items	Lambda	Scan / Query
GET	/items/{id}	Get item by ID	Lambda	GetItem
POST	/items	Create new item	Lambda	PutItem
PUT	/items/{id}	Update item	Lambda	UpdateItem
DELETE	/items/{id}	Delete item	Lambda	DeleteItem

BEST PRACTICES

- ✓ Use IAM roles, never hardcode credentials.
- ✓ Validate and sanitize all inputs.
- ✓ Keep Lambda functions small and focused.
- ✓ Use meaningful API responses.
- ✓ Monitor with CloudWatch Logs and Metrics.
- ✓ Use DynamoDB with proper partition keys.



JUNIOR ENGINEER TIP

Serverless does not mean "no operations". You still need to think about security, permissions, monitoring, errors, and business logic.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SERVERLESS. SCALABLE. SECURE. SIMPLE.

PROJECT 11

DESIGN DYNAMODB FOR SCALE

BUILD HIGH-PERFORMANCE, COST-EFFECTIVE, AND SCALABLE DATA MODELS

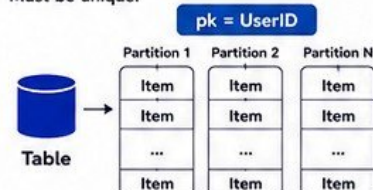
WHAT YOU WILL BUILD

- ✓ Design tables using access patterns
- ✓ Use partition and sort keys effectively
- ✓ Build Global Secondary Indexes (GSI)
- ✓ Choose On-Demand vs Provisioned capacity
- ✓ Use DynamoDB Streams
- ✓ Identify and solve hot partition issues
- ✓ Avoid Scan-heavy and inefficient designs

DYNAMODB KEY CONCEPTS

PARTITION KEY (HASH KEY)

Distributes data across partitions.
Must be unique.



SORT KEY (RANGE KEY)

Sorts items within the same partition.
Enables efficient queries.

pk = UserID	sk = Timestamp
UserID	Timestamp
U1001	2024-01-01 10:00:00
U1001	2024-01-01 11:00:00
U1001	2024-01-01 12:00:00
...	

ACCESS-PATTERN-FIRST DESIGN

- 1 Identify all access patterns.
- 2 Example: Get all orders for a user.
- 3 Design keys to support queries (not scans).
- 4 One access pattern = one table or index.
- 5 Denormalize data for speed.

DESIGN FOR QUERIES, NOT FOR NORMALIZATION.

GLOBAL SECONDARY INDEXES (GSI)

Allow queries on attributes that are not the primary key.

GSI Example

GSI Name : ByStatus
Partition Key : Status
Sort Key : CreatedAt
Use Case : Query orders by status

ON-DEMAND vs PROVISIONED

	ON-DEMAND	PROVISIONED
Capacity	Auto scales	Manual settings
Best For	Unpredictable workloads	Predictable workloads
Billing	Per request	Per capacity unit
Scaling	Instant	Manual
Cost	Higher per request	Lower at scale

DYNAMODB STREAMS

Capture changes to items in your table in near real-time.

Use Cases:

- Trigger Lambdas
- Real-time analytics
- Replication
- Audit logging

Streams are ordered per partition key.

HOT PARTITION PROBLEMS



Too many writes/reads on a single partition key = throttling.

Causes:

- Skewed key distribution
- Monotonic keys (e.g., timestamps)
- Low cardinality keys

Solutions:

- Add random suffix/prefix
- Use UUIDs
- Shard the key

AVOID SCAN-HEAVY DESIGNS



Scan reads every item = expensive and slow.

Why it's bad:

- High RCUs
- Slow performance
- Hard to scale

Use Query, not Scan.

KEY DESIGN EXAMPLE

Use Case: Get all orders for a user in a date range.

TABLE DESIGN

Table Name	Orders
Partition Key	userId (String)
Sort Key	orderDate (String)
Attributes	orderId, status, total, items, ...

ACCESS PATTERNS

- ✓ Get all orders for a user (by date range)
- ✓ Get a single order by user + orderId
- ✓ Query recent orders for a user

Efficient: Uses Query

CAPACITY MODES EXAMPLE



ON-DEMAND (Good for New/Variable Apps)

- No capacity planning
- Auto scales instantly
- Pay per request



PROVISIONED (Good for Steady Workloads)

- Set RCUs/WCUs
- Predictable cost
- Auto Scaling available

DESIGN CHECKLIST



Start with access patterns



Choose the right partition and sort keys



Create GSIs for alternate access patterns



Choose the right capacity mode



Enable Streams if needed



Avoid hot partitions



Avoid Scan, use Query

COMMON ANTI-PATTERNS

- ✗ Using Scan to fetch data
- ✗ Using low-cardinality keys
- ✗ Storing relational data as-is
- ✗ Ignoring hot partition risk
- ✗ Over-fetching large items



JUNIOR ENGINEER TIP

Model your data for your queries, not your ER diagram.
A good DynamoDB design is simple, fast, and predictable.

BEST PRACTICES

- ✓ Keep items small (< 400 KB)
- ✓ Use TTL for automatic expiration
- ✓ Monitor with CloudWatch metrics
- ✓ Use Projection in GSIs
- ✓ Test with realistic data
- ✓ Review access patterns regularly



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

RIGHT MODEL. RIGHT DESIGN. RIGHT SCALE.

PROJECT 12

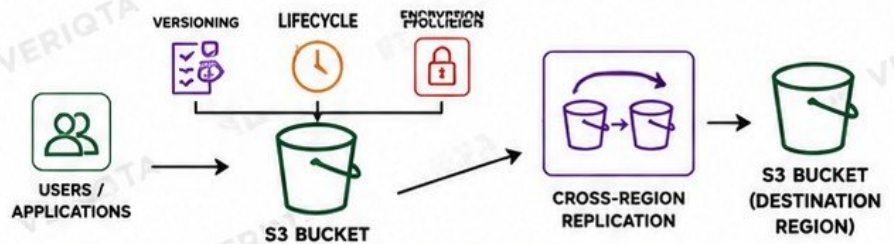
CREATE A RESILIENT FILE STORAGE ARCHITECTURE

STORE DATA DURABLY, ACCESSIBLY, AND COST-EFFECTIVELY

WHAT YOU WILL BUILD

- ✓ Understand S3 storage classes
- ✓ Enable versioning for protection
- ✓ Configure lifecycle policies
- ✓ Set up Cross-Region Replication
- ✓ Encrypt data at rest and in transit
- ✓ Use EFS for shared file access
- ✓ Use EBS for block storage needs
- ✓ Choose the right storage for the job

RESILIENT S3 ARCHITECTURE



Highly durable • Cost optimized • Encrypted • Geo-redundant

S3 STORAGE CLASSES (CHOOSE BASED ON ACCESS PATTERN)

S3 STANDARD	S3 STANDARD IA	S3 ONE ZONE-IA	S3 GLACIER INSTANT RETRIEVAL	S3 GLACIER FLEXIBLE RETRIEVAL	S3 GLACIER DEEP ARCHIVE
					
Frequent access Low latency High throughput	Infrequent access Lower cost Retrieval fee	Infrequent access Stored in one AZ Lower cost	Long term archive Milliseconds retrieval Lower cost	Minutes to hours retrieval Lower cost	Lowest cost Hours retrieval Long term archive
Use for: Active data, web content, apps	Use for: Backups, long-tail data	Use for: Recreateable data, non-critical	Use for: Long term archive, compliance	Use for: Archives not needed quickly	Use for: Deep archive, compliance

S3 KEY FEATURES

- VERSIONING**
Protect against accidental deletions and overwrites.
- LIFECYCLE POLICIES**
Move data between storage classes automatically.
- CROSS-REGION REPLICATION**
Replicate objects to another region for DR and compliance.
- ENCRYPTION**
Use SSE-S3, SSE-KMS, or Bucket Keys for security.

SHARED FILE SYSTEM: AMAZON EFS

- Managed NFS file system
- Shared across many EC2 instances
- Scales automatically
- Multi-AZ for high availability
- Use for: Lift-and-shift apps, content management, home directories, shared application data

BLOCK STORAGE: AMAZON EBS

- Block-level storage for EC2
- Attached to one instance at a time
- High performance
- Different volume types (gp3, io1, st1, sc1, etc.)
- Use for: Databases, boot volumes, transactional applications

S3 vs EFS vs EBS – WHEN TO USE WHAT?

STORAGE TYPE	ACCESS PATTERN	PROTOCOL	SHARED?	DURABILITY	BEST FOR	NOT IDEAL FOR
S3	Object storage (anywhere)	HTTP / HTTPS	Yes (global)	11 9's (99.999999999%)	Backups, static files, logs, archives, media, data lake	Low latency app data, databases, boot drives
EFS	File storage (shared)	NFS	Yes (multi-AZ)	11 9's	Shared app data, content management, home directories	High IOPS transactional workloads
EBS	Block storage (single instance)	Block	No (single EC2)	Single-AZ (Multi-AZ with snapshots)	Databases, boot volumes, apps needing low latency	Shared access, multi-AZ native



DESIGN FOR DURABILITY, SECURITY, AVAILABILITY, AND COST.
THE RIGHT STORAGE ARCHITECTURE IS THE FOUNDATION OF A RESILIENT APPLICATION.



JUNIOR ENGINEER TIP

Always match the storage solution to the workload. Wrong choice leads to high cost, poor performance, or operational headaches. Understand the access pattern, not just the service name.

PROJECT 13

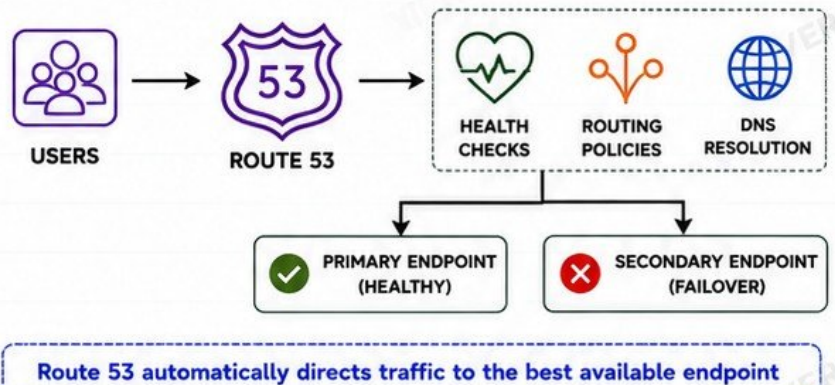
BUILD DNS FAILOVER WITH ROUTE 53

HIGHLY AVAILABLE DNS FOR RESILIENT APPLICATIONS

WHAT YOU WILL BUILD

- ✓ Create a hosted zone in Route 53
- ✓ Configure alias records
- ✓ Set up health checks
- ✓ Implement failover routing
- ✓ Implement weighted routing
- ✓ Implement latency-based routing
- ✓ Understand TTL behavior
- ✓ Simulate an unhealthy endpoint and observe failover

HIGH LEVEL ARCHITECTURE



ROUTING POLICIES OVERVIEW

SIMPLE ROUTING



Default routing policy. Use when you have one resource.

FAILOVER ROUTING



Active-passive configuration. Traffic fails over to backup when primary is unhealthy.

WEIGHTED ROUTING



Distribute traffic across resources based on assigned weights.

LATENCY-BASED ROUTING



Route users to the endpoint with the lowest network latency.

GEOLOCATION ROUTING



Route traffic based on the geographic location of users.

MULTIVALUE ANSWER ROUTING



Return multiple healthy records (up to 8) for high availability.

TTL (TIME TO LIVE) BEHAVIOR

TTL VALUE	EFFECT
Low TTL (e.g. 30s)	Changes propagate quickly, but more DNS queries (higher cost).
Moderate TTL (e.g. 60-300s)	Balanced propagation speed and DNS query load.
High TTL (e.g. 3600s)	Changes take longer to propagate, but reduces DNS query load.



JUNIOR ENGINEER TIP

Start with a low TTL (30-60s) when testing. After validation, you can increase it (300-900s) for better performance and lower DNS cost.

SIMULATE AN UNHEALTHY ENDPOINT

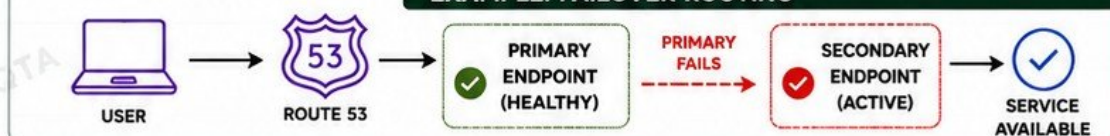
- 1 Create a health check for your primary endpoint.
- 2 Configure failover routing with primary (active) and secondary (passive).
- 3 Verify traffic goes to the primary endpoint.
- 4 Make the primary endpoint unhealthy (stop the service or block the health check path).
- 5 Observe Route 53 automatically routing traffic to the secondary endpoint.
- 6 Fix the primary endpoint and watch traffic return to it automatically.

HEALTH CHECKS



Continuously monitor your endpoints. If a health check fails the configured threshold, Route 53 stops sending traffic to it.

EXAMPLE: FAILOVER ROUTING



PROJECT 14

CONNECT PRIVATE NETWORKS SECURELY

DESIGN PRIVATE CONNECTIVITY BETWEEN AWS NETWORKS AND ON-PREMISES

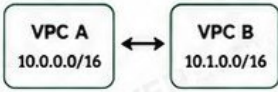
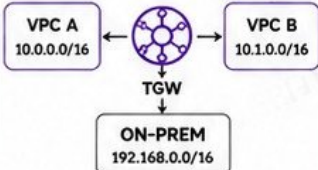
WHAT YOU WILL BUILD

- ✓ Connect VPCs using VPC Peering
- ✓ Centralize connectivity with Transit Gateway
- ✓ Connect on-premises using Site-to-Site VPN
- ✓ Understand AWS Direct Connect concepts
- ✓ Enable route propagation
- ✓ Handle CIDR overlap challenges
- ✓ Make the right connectivity decisions
- ✓ Troubleshoot missing routes

PRIVATE CONNECTIVITY OPTIONS OVERVIEW

VPC PEERING	TRANSIT GATEWAY	SITE-TO-SITE VPN	AWS DIRECT CONNECT
			
VPC ↔ VPC	VPCs ↔ TGW ↔ VPCs	AWS ↔ ON-PREM	AWS ↔ ON-PREM
Simple, low cost Direct connection between two VPCs	Central hub for many VPCs and on-prem	Encrypted VPN tunnel over the Internet	Dedicated private network connection
Best for: Few VPCs, no transitive routing	Best for: Many VPCs, hybrid environments	Best for: Cost-effective hybrid connectivity	Best for: High bandwidth, low latency, stable

CONNECTIVITY ARCHITECTURE EXAMPLES

VPC PEERING EXAMPLE	TRANSIT GATEWAY EXAMPLE	SITE-TO-SITE VPN EXAMPLE	DIRECT CONNECT EXAMPLE
			
Routes must be added in both VPC route tables	Centralized routing and policy control	Encrypted tunnel over the public Internet	Private, dedicated, high performance link

ROUTE PROPAGATION

- ✓ Automatic route sharing between connected networks.
- ✓ With TGW, enable route propagation from attachments.
- ✓ For VPC Peering and VPN, routes are not propagated automatically.



CIDR OVERLAP CHALLENGE

Overlapping CIDR blocks between networks cause routing conflicts.



Solutions:

- Re-design CIDR blocks to be unique
- Use NAT, proxies, or network segmentation
- Use TGW with different CIDRs

PRIVATE CONNECTIVITY DECISIONS

SCENARIO	BEST OPTION
Connect two VPCs	VPC Peering
Many VPCs + On-Prem	Transit Gateway
Low-cost hybrid connectivity	Site-to-Site VPN
High performance, consistent bandwidth	AWS Direct Connect

TROUBLESHOOTING MISSING ROUTES

- 1 Verify route exists in the route table.
- 2 Check if the route table is associated with the subnet.
- 3 Ensure the connection (Peering/VPN/DX/TGW) is active.
- 4 For TGW, verify route propagation is enabled.
- 5 Check security groups and NACLs (return traffic).
- 6 For VPN/DX, verify tunnels/connections are up.
- 7 Use Reachability Analyzer to validate the path.

USEFUL TOOLS



- VPC Reachability Analyzer
- Route Table Viewer
- Network Path Insights
- CloudWatch Metrics
- VPC Flow Logs

KEY TAKEAWAYS



Choose the right connectivity option for your scale, cost, and performance needs.



Understand routing behavior and propagation.



Avoid CIDR overlap issues before they impact routing.



Always troubleshoot logically when routes are missing.



JUNIOR ENGINEER TIP

Good network design is the foundation of secure, reliable, and scalable cloud architectures.

PLAN CIDRS CAREFULLY.
ROUTING IS EVERYTHING!

PROJECT 15

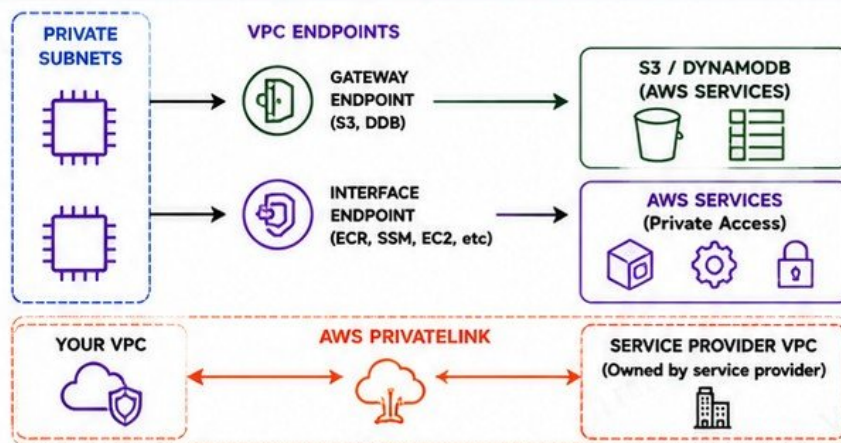
BUILD PRIVATE AWS SERVICE ACCESS

KEEP TRAFFIC PRIVATE, SECURE, AND COST-EFFECTIVE

WHAT YOU WILL BUILD

- ✓ Create Gateway VPC endpoints
- ✓ Create Interface VPC endpoints
- ✓ Understand AWS PrivateLink
- ✓ Enable private S3 access
- ✓ Configure endpoint policies
- ✓ Understand DNS considerations
- ✓ Reduce unnecessary NAT traffic
- ✓ Verify traffic remains private

PRIVATE ACCESS ARCHITECTURE OVERVIEW



VPC ENDPOINT TYPES

GATEWAY ENDPOINT



- Attached to route table
- Supports S3 and DynamoDB
- No hourly charge
- Uses AWS backbone
- Ideal for large data transfer

INTERFACE ENDPOINT



- Deployed in subnets with ENIs
- Supports many AWS services
- Hourly charge per AZ
- Private IP addresses
- Use for AWS services other than S3/DynamoDB

AWS PRIVATELINK



- Connect to services owned by other AWS accounts or SaaS providers
- Uses interface endpoints
- Private, secure, scalable
- No traffic over the public Internet

PRIVATE S3 ACCESS



EC2 INSTANCE
(Private Subnet)



S3 GATEWAY
ENDPOINT



S3 BUCKET
(Private Access)



Traffic stays within the AWS network. No Internet Gateway or NAT Gateway required.

ENDPOINT POLICIES



- Add resource policies to endpoints
- Control which IAM principals can access
- Restrict to specific buckets, actions, or prefixes
- Adds an extra layer of security

DNS CONSIDERATIONS



- Private DNS: Use AWS private DNS to resolve AWS service hostnames to private IPs
- Ensure "Enable Private DNS name" is checked for interface endpoints
- For S3, use the regional endpoint (e.g., s3.us-east-1.amazonaws.com)

REDUCE UNNECESSARY NAT GATEWAY TRAFFIC



- NAT Gateways are expensive
- Use VPC endpoints for AWS service access
- Keep traffic on the AWS backbone
- Lower costs and improve performance

HOW TO VERIFY TRAFFIC REMAINS PRIVATE

- 1 Ensure route table for subnets points to the correct endpoint (Gateway Endpoint).
- 2 For interface endpoints, verify the ENI IPs are used (not public IPs).
- 3 Use VPC Flow Logs to confirm traffic goes to endpoint ENIs or AWS service IP ranges.
- 4 Run curl or aws cli commands and check there is no Internet route.
- 5 Ensure security groups allow outbound to endpoint or service as needed.
- 6 Confirm no data is going through NAT Gateway using CloudWatch metrics.



JUNIOR ENGINEER TIP

VPC Endpoints = Private, Secure, and Cost-Effective.

Design for Private by Default!



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PROJECT 16

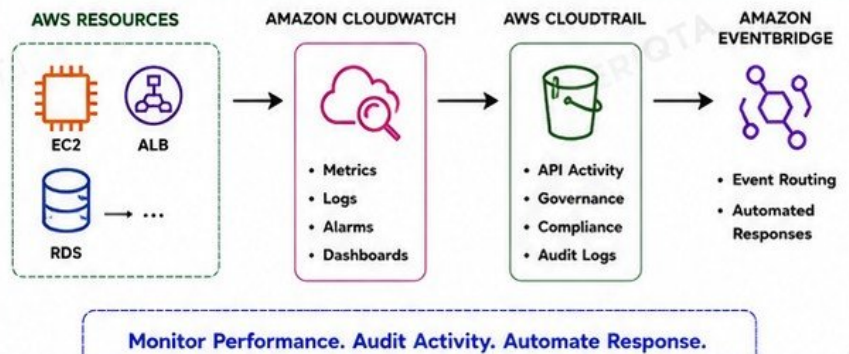
MONITOR AND DIAGNOSE AN AWS APPLICATION

OBSERVE, ALERT, AND ACT BEFORE USERS ARE IMPACTED

WHAT YOU WILL BUILD

- ✓ Monitor AWS resources with CloudWatch
- ✓ Collect and analyze metrics
- ✓ Centralize and search logs
- ✓ Create alarms and notifications
- ✓ Audit API activity with CloudTrail
- ✓ React to events with EventBridge
- ✓ Monitor EC2 and ALB performance
- ✓ Diagnose high CPU, unhealthy targets, and application errors
- ✓ Separate monitoring from auditing

OBSERVABILITY STACK OVERVIEW



CLOUDWATCH – THE CORE OBSERVABILITY SERVICE

METRICS



- Collect time-series data
- Monitor performance
- Built-in and custom metrics
- High resolution (1s)

LOGS



- Centralize logs from AWS services
- CloudWatch Logs Insights queries
- Search and analyze application behavior

ALARMS



- Set thresholds
- Trigger actions
- Send notifications (SNS, Email, ChatOps)
- Prevent or reduce incidents

DASHBOARDS



- Visualize metrics
- Real-time overview
- Share with teams
- Custom widgets and views

KEY AWS SERVICES



CLOUDWATCH
Monitoring, metrics, logs, alarms, dashboards.



CLOUDTRAIL
Audit API calls and user activity across AWS.



EVENTBRIDGE
React to state changes and service events.



SNS
Deliver alerts and notifications.

MONITOR EC2 AND ALB – KEY METRICS

EC2 METRICS



- CPUUtilization
- StatusCheckFailed
- NetworkIn / NetworkOut
- DiskReadOps / DiskWriteOps
- DiskSpaceUtilization
- MemoryUtilization (via CloudWatch Agent)

ALB METRICS



- RequestCount
- TargetResponseTime
- HTTPCode_ELB_4XX_Count
- HTTPCode_ELB_5XX_Count
- HealthyHostCount
- UnHealthyHostCount

COMMON ISSUES AND HOW TO DIAGNOSE

ISSUE	WHAT IT LOOKS LIKE	HOW TO DIAGNOSE
HIGH CPU	CPUUtilization > 80%	Check CloudWatch Metrics, EC2 instance type, scale out if needed.
UNHEALTHY TARGETS	ALB reports UnHealthyHostCount > 0	Check target health, security groups, health check path, app logs.
APPLICATION ERRORS	5XX errors, timeouts, or high latency	Check ALB 5XX metrics, application logs, backend dependencies.

AUDITING vs MONITORING

MONITORING (CLOUDWATCH)



- Focused on performance and availability
- Metrics, logs, alarms, dashboards
- Helps you OPERATE better
- Example: CPU high, response time slow



AUDITING (CLOUDTRAIL)

- Focused on API activity and user actions
- Records who did what, when, and from where
- Helps you be COMPLIANT
- Example: Policy changed, key deleted

TROUBLESHOOTING WORKFLOW

- 1 Detect: Alarm fires or issue reported.
- 2 Observe: Check CloudWatch dashboards and metrics.
- 3 Investigate: Analyze logs (CloudWatch Logs Insights).
- 4 Correlate: Review ALB + EC2 metrics together.
- 5 Audit: Use CloudTrail to investigate user/API changes.
- 6 Act: Fix issue or roll back changes.
- 7 Automate: Use EventBridge + SNS/ChatOps for alerts and runbooks.



JUNIOR ENGINEER TIP

Always monitor the Golden Signals:
Latency, Traffic, Errors, Saturation.
Know your baselines and alert on deviations.



BEST PRACTICES

- Set meaningful alarms, not just defaults.
- Centralize logs and use retention policies.
- Use dashboards for visibility and quick decisions.
- Review and test alarms regularly.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PROJECT 17

DESIGN BACKUP AND DISASTER RECOVERY

PROTECT YOUR DATA. REDUCE DOWNTIME. RECOVER WITH CONFIDENCE.

KEY CONCEPTS

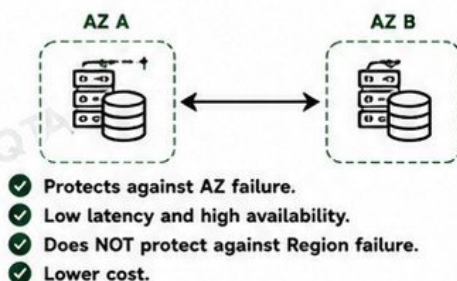
-  **RTO (RECOVERY TIME OBJECTIVE)**
How fast you must recover after a failure.
-  **RPO (RECOVERY POINT OBJECTIVE)**
How much data loss you can tolerate.
-  **BACKUP**
Copy of data for point-in-time recovery.
-  **DISASTER RECOVERY**
Process of restoring systems after a major disruption.

AWS BACKUP TOOLS

-  **AWS BACKUP**
Centralized service to automate and manage backups across AWS services.
-  **RDS SNAPSHOTS**
Point-in-time backups stored in S3. Automated or manual.
-  **S3 REPLICATION**
Cross-Region Replication (CRR) protects against Region failure.
-  **AWS BACKUP VAULT**
Store and protect backups with retention and audit controls.

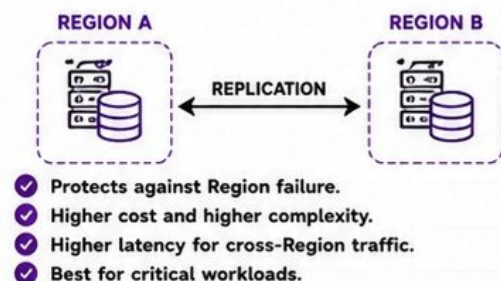
MULTI-AZ vs MULTI-REGION

MULTI-AZ (WITHIN A REGION)



VS

MULTI-REGION (ACROSS REGIONS)



DISASTER RECOVERY STRATEGIES

-  **PILOT LIGHT**
Minimal core resources running. Scale up on demand. **LOWEST**
-  **WARM STANDBY**
Reduced-size environment running. Scale up quickly. **MEDIUM**
-  **ACTIVE / ACTIVE**
Full environments running in multiple Regions. **HIGHEST**

BACKUP AND RESTORE FLOW



CHOOSE A STRATEGY BASED ON BUSINESS NEEDS

REQUIREMENT	RTO	RPO	SUGGESTED STRATEGY
Non-critical workloads / Dev	Hours	Hours	Pilot Light
Business applications	Minutes	Minutes	Warm Standby
Mission-critical applications	Seconds	Seconds	Active / Active



JUNIOR ENGINEER TIP

Backups are important, but untested backups are useless.
Always test your restore process before you need it in a real disaster!

PROJECT 18

IMPROVE SECURITY AT THE APPLICATION EDGE

Secure traffic before it reaches your application.

KEY SERVICES



CLOUDFRONT

Global CDN that caches content and accelerates delivery.



AWS WAF

Web application firewall that filters malicious traffic.



AWS SHIELD

DDoS protection for your applications.



HTTPS

Encrypts traffic in transit using TLS.



ACM

Provision and manage SSL/TLS certificates easily.



ALB

Distributes traffic to your application securely.

LAYERED SECURITY APPROACH



PROTECT AT THE EDGE

CloudFront + AWS WAF block common attacks, malicious bots, and unwanted traffic.



MITIGATE DDoS ATTACKS

AWS Shield provides managed DDoS protection at the network and transport layers.



ENCRYPT TRAFFIC

Use HTTPS with TLS certificates from ACM to encrypt all traffic end-to-end.



ROUTE SECURELY

ALB handles traffic securely and forwards to healthy application instances.



RESTRICT ORIGIN EXPOSURE

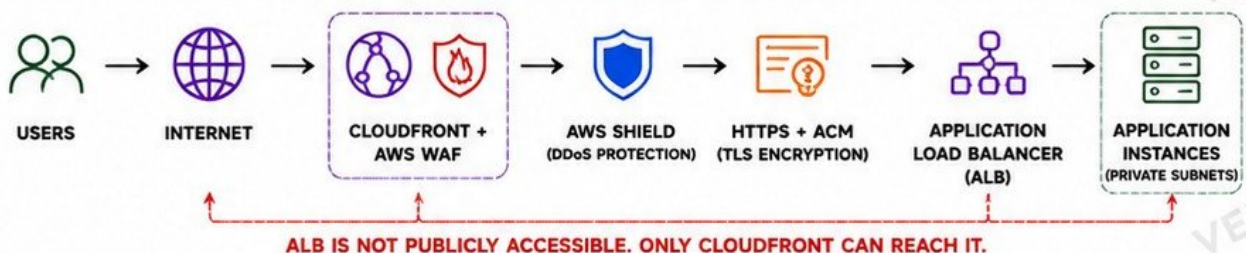
Keep your origin (ALB) private. Only allow traffic from CloudFront using security groups.



DEFENSE IN DEPTH

Multiple layers ensure your application stays secure even if one layer is bypassed.

SECURE APPLICATION EDGE ARCHITECTURE



BEST PRACTICES

- ✓ Use WAF managed rules and custom rules.
- ✓ Enable AWS Shield Standard (always on).
- ✓ Use ACM certificates and redirect HTTP to HTTPS.
- ✓ Restrict ALB security group to allow traffic only from CloudFront IP ranges.
- ✓ Monitor with CloudWatch, WAF logs, and Shield metrics.

BENEFITS

- ✓ Blocks common web exploits and bots.
- ✓ Protects against DDoS attacks.
- ✓ Encrypts traffic for confidentiality.
- ✓ Hides your origin infrastructure.
- ✓ Improves performance and reliability.



JUNIOR ENGINEER TIP

Security at the edge is the first line of defense.
Always combine CloudFront, WAF, Shield, HTTPS, and restricted origins.

PROJECT 19

FIX AN EXPENSIVE AWS ARCHITECTURE

REDUCE COSTS WITHOUT SACRIFICING RELIABILITY OR PERFORMANCE.

WHAT YOU WILL DO

- ✓ Examine an intentionally inefficient design
- ✓ Identify hidden and unnecessary costs
- ✓ Apply AWS best practices to optimize
- ✓ Balance cost, reliability, performance, and operational effort
- ✓ Re-architect for efficiency
- ✓ Measure savings and impact

COMMON COST DRIVERS



NAT GATEWAY COSTS
High data processing charges and hourly cost.



EC2 OVERPROVISIONING
Instances are larger than needed.



UNUSED EBS VOLUMES
Orphaned volumes accumulate costs.



S3 STORAGE CLASSES
Using Standard when cheaper tiers fit.



AUTO SCALING
Not enabled or not properly configured.



SAVINGS PLANS
Missing opportunities for committed usage.

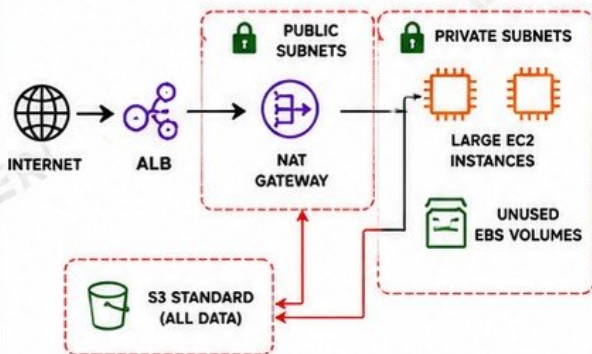


CLOUDFRONT CACHING
Content not cached, increasing origin cost.



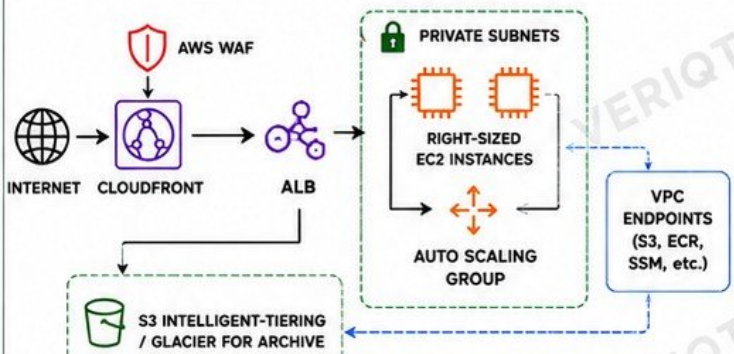
VPC ENDPOINTS
Traffic to AWS services going through NAT.

BEFORE: EXPENSIVE ARCHITECTURE (INTENTIONALLY INEFFICIENT)



HIGH COSTS DUE TO UNNECESSARY TRAFFIC, OVERSIZED RESOURCES, AND MISCONFIGURATIONS.

AFTER: OPTIMIZED ARCHITECTURE



LOWER COSTS, HIGHER EFFICIENCY, BETTER PERFORMANCE.

COST OPTIMIZATION PRACTICES

- ✓ Replace NAT Gateway with VPC Endpoints for AWS services.
- ✓ Right-size EC2 instances using CloudWatch metrics.
- ✓ Delete unused EBS volumes and snapshots.
- ✓ Use S3 Intelligent-Tiering, Standard-IA, and Glacier.
- ✓ Enable Auto Scaling to match demand.
- ✓ Use Savings Plans or Reserved Instances for steady workloads.
- ✓ Leverage CloudFront caching to offload origins.
- ✓ Monitor and alert on cost anomalies.

AWS TOOLS THAT HELP

- AWS Cost Explorer** – Analyze and understand costs.
- AWS Budgets** – Set budgets and alerts.
- AWS Trusted Advisor** – Get cost optimization recommendations.
- AWS Compute Optimizer** – Rightsize your resources.
- CloudWatch** – Monitor usage and performance.

BALANCE THE FOUR PILLARS



RELIABILITY
Maintain high availability and fault tolerance.



PERFORMANCE
Deliver low latency and fast responses.



COST
Pay only for what you use.



OPERATIONAL EFFORT
Keep architecture simple, manageable, and scalable.



JUNIOR ENGINEER TIP

The cheapest architecture is not always the best.
Optimize with a balance of cost, reliability, performance, and operational efficiency.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

FINAL PROJECT

ARCHITECT A PRODUCTION AWS APPLICATION

COMBINE EVERYTHING. BUILD IT RIGHT. SECURE IT. SCALE IT. MONITOR IT.



**COMBINE THE
ENTIRE HANDBOOK**
All concepts in one
production-grade
architecture.



**INTRODUCE FAILURES
AND TROUBLESHOOT**
Test resilience.
Break things.
Fix them.

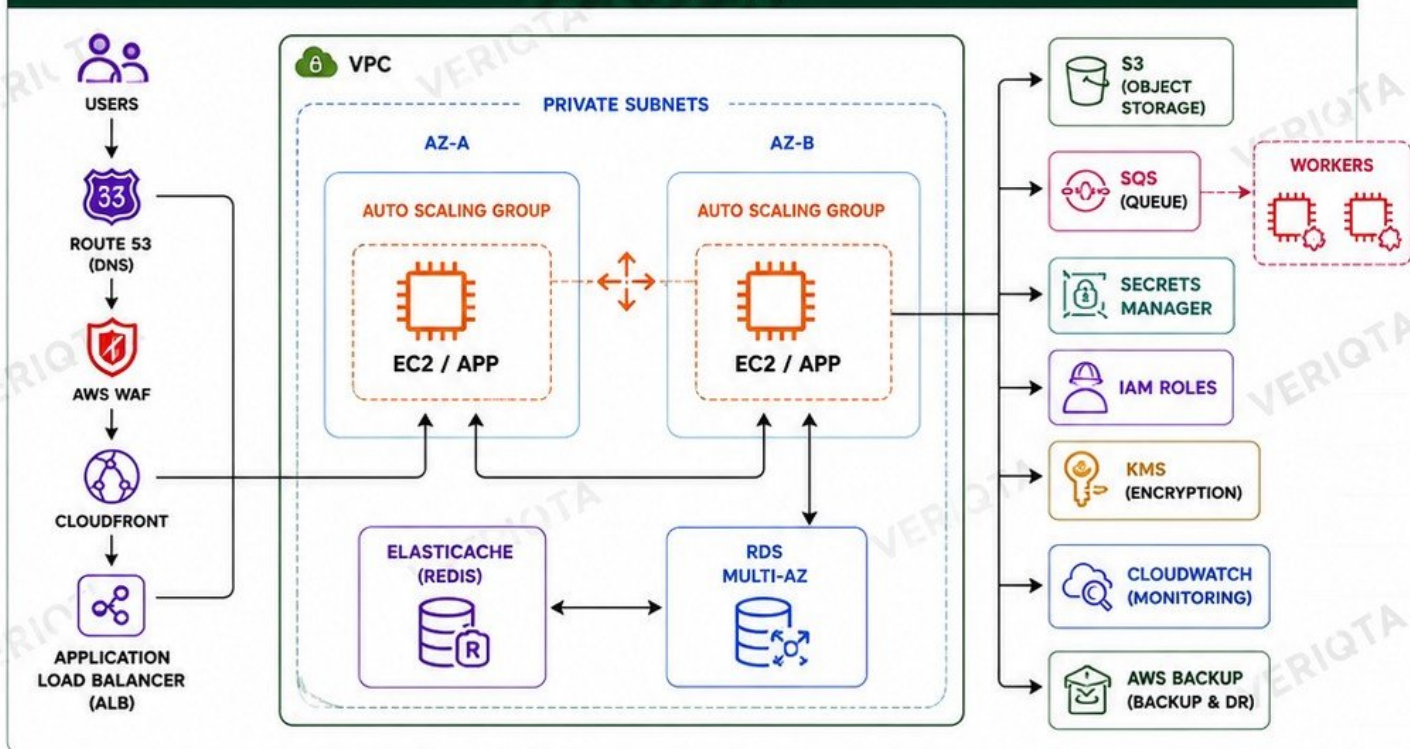


**EXPLAIN EVERY
DECISION**
Use SAA-C03
principles for each
architecture choice.



BALANCE THE 4 PILLARS
✓ Cost
✓ Reliability
✓ Performance
✓ Operational Excellence

FINAL ARCHITECTURE



KEY COMPONENTS

- ✓ Route 53 for reliable DNS.
- ✓ CloudFront + WAF for performance and protection.
- ✓ ALB for secure traffic distribution.
- ✓ Multi-AZ private app tier with Auto Scaling.
- ✓ ElastiCache for caching and performance.
- ✓ RDS Multi-AZ for high availability.
- ✓ S3 for durable object storage.
- ✓ SQS decouples workloads.
- ✓ Secrets Manager for secure credentials.
- ✓ IAM roles for least privilege access.
- ✓ KMS for encryption.
- ✓ CloudWatch for observability.
- ✓ AWS Backup for data protection.

TEST FAILURES & RECOVERY

- ❗ Simulate instance failure → Auto Scaling replaces.
- ❗ Make an AZ unavailable → traffic shifts to healthy AZ.
- ❗ Break the application → use CloudWatch to diagnose.
- ❗ Fill the queue → verify workers scale.
- ❗ Lose database primary → Multi-AZ failover.
- ❗ Revoke permissions → verify IAM policies.
- ❗ Restore from backup → recover data.



JUNIOR ENGINEER TIP

Architecture is not just drawing boxes.
It's making the right trade-offs based on
business requirements and SAA-C03 principles.

SAA-C03 PRINCIPLES APPLIED

- ✓ Design for High Availability (Multi-AZ, Auto Scaling, Health Checks)
- ✓ Design for Durability (S3, RDS Multi-AZ, Backup)
- ✓ Design for Security (WAF, IAM, KMS, Secrets Manager)
- ✓ Design for Performance Efficiency (CloudFront, ElastiCache, ALB)
- ✓ Design for Cost Optimization (Auto Scaling, Right-sizing, S3 Classes, Savings Opportunities)
- ✓ Design for Operational Excellence (CloudWatch, IAM, Backup, Automation)